

✓ Problem 2.a: Feature Extraction (30 points)

Use pretrained ResNet as feature extractor + Train traditional ML classifier

Dataset: UCMerced - 21 classes, download from <http://weegee.vision.ucmerced.edu/datasets/landuse.html>

Steps:

1. Load ResNet, remove FC layer
2. Extract features for all images
3. Train XGBoost/SVM
4. Report: Accuracy, Precision, Recall, F1, Confusion Matrices

✓ 1. Install Required Packages

First, let's install all the packages we'll need for this problem.

```
%pip install torch torchvision numpy pandas scikit-learn xgboost matplotlib seaborn Pillow tqdm --quiet
```

✓ 2. Import Libraries and Setup

Import all necessary libraries and set up our environment with reproducible random seeds.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')

import torch
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, classification_re
from xgboost import XGBClassifier

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Set random seeds for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
Using device: cuda
GPU: Tesla T4
```

✓ 3. Set Data Path

IMPORTANT - READ BEFORE RUNNING:

For Google Colab (RECOMMENDED):

1. **Uncomment** lines 1-3 (the Colab section)
2. **Comment out** line 6 (the local path)
3. Make sure your dataset is uploaded to the specified Drive path

For Local Execution:

1. Keep line 6 as is (local path)
2. Lines 1-3 should remain commented out

```
# For Google Colab (uncomment these lines if using Colab):
from google.colab import drive
drive.mount('/content/drive')
DATA_ROOT = '/content/drive/MyDrive/IMG589/HW5/UCMerced_LandUse/Images'

## For Local (use this for now):
# DATA_ROOT = '/Users/varagantibasanthkumar/Desktop/ IMG5 89/-Applications-of-Machine-learning-in-Remote-Sensin

# Verify path exists
if not os.path.exists(DATA_ROOT):
    raise FileNotFoundError(f"Data directory not found: {DATA_ROOT}")

print(f"Data directory: {DATA_ROOT}")
print(f"Classes found: {len(os.listdir(DATA_ROOT))}")

Mounted at /content/drive
Data directory: /content/drive/MyDrive/IMG589/HW5/UCMerced_LandUse/Images
Classes found: 21
```

4. Load and Explore Dataset

Load all image paths and labels, then perform exploratory data analysis.

```
def load_dataset_paths(root_dir):
    """
    Load all image paths and their corresponding labels.

    Args:
        root_dir: Root directory containing class subdirectories

    Returns:
        image_paths: List of full paths to images
        labels: List of class labels
        class_names: List of unique class names
    """
    image_paths = []
    labels = []
    class_names = sorted(os.listdir(root_dir))

    # Remove any hidden files
    class_names = [c for c in class_names if not c.startswith('.')]

    for class_name in class_names:
        class_dir = os.path.join(root_dir, class_name)
        if not os.path.isdir(class_dir):
            continue

        for img_name in os.listdir(class_dir):
            if img_name.endswith(('.tif', '.jpg', '.png')):
                image_paths.append(os.path.join(class_dir, img_name))
                labels.append(class_name)

    return image_paths, labels, class_names

# Load dataset
image_paths, labels, class_names = load_dataset_paths(DATA_ROOT)

print(f"Total images: {len(image_paths)}")
print(f"Total classes: {len(class_names)}")
print(f"\nClass names:")
for i, cls in enumerate(class_names, 1):
    count = labels.count(cls)
    print(f" {i:2d}. {cls:20s} - {count} images")
```

```
Total images: 2100
Total classes: 21

Class names:
```

1. agricultural	- 100 images
2. airplane	- 100 images
3. baseballdiamond	- 100 images
4. beach	- 100 images
5. buildings	- 100 images
6. chaparral	- 100 images
7. denseresidential	- 100 images
8. forest	- 100 images
9. freeway	- 100 images
10. golfcourse	- 100 images
11. harbor	- 100 images
12. intersection	- 100 images
13. mediumresidential	- 100 images
14. mobilehomepark	- 100 images
15. overpass	- 100 images
16. parkinglot	- 100 images
17. river	- 100 images
18. runway	- 100 images
19. sparseresidential	- 100 images
20. storagetanks	- 100 images
21. tenniscourt	- 100 images

✓ 4.1 Visualize Sample Images

Let's visualize a few sample images from different classes to understand our dataset better.

```
# Visualize sample images from each class
fig, axes = plt.subplots(3, 7, figsize=(20, 9))
axes = axes.flatten()

for idx, class_name in enumerate(class_names):
    # Get first image from this class
    class_images = [p for p, l in zip(image_paths, labels) if l == class_name]
    img_path = class_images[0]

    img = Image.open(img_path).convert('RGB')
    axes[idx].imshow(img)
    axes[idx].set_title(class_name, fontsize=11, fontweight='bold')
    axes[idx].axis('off')

plt.suptitle('Sample Images from Each Class (UCMerced Land Use Dataset)',
            fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()
```



5. Data Splitting

Split the dataset into train (70%), validation (15%), and test (15%) sets with stratification to maintain class balance.

```
# Encode labels
label_encoder = LabelEncoder()
labels_encoded = label_encoder.fit_transform(labels)

# First split: 70% train, 30% temp (val + test)
X_train, X_temp, y_train, y_temp = train_test_split(
    image_paths, labels_encoded,
    test_size=0.30,
    random_state=SEED,
    stratify=labels_encoded
)

# Second split: Split temp into 50-50 for val and test (15% each of total)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp
)

print(f"Dataset splits:")
print(f"  Train:      {len(X_train):4d} images ({len(X_train)/len(image_paths)*100:.1f}%)")
print(f"  Validation: {len(X_val):4d} images ({len(X_val)/len(image_paths)*100:.1f}%)")
print(f"  Test:       {len(X_test):4d} images ({len(X_test)/len(image_paths)*100:.1f}%)")
print(f"  Total:      {len(image_paths):4d} images")

# Verify class distribution
```

```
print(f"\nClass distribution in each split:")
print(f"  Train:      {np.bincount(y_train)}")
print(f"  Validation: {np.bincount(y_val)}")
print(f"  Test:       {np.bincount(y_test)}")
```

Dataset splits:

```
Train: 1470 images (70.0%)
Validation: 315 images (15.0%)
Test: 315 images (15.0%)
Total: 2100 images
```

Class distribution in each split:

```
Train: [70 70 70 70 70 70 70 70 70 70 70 70 70 70 70 70 70 70 70 70]
Validation: [15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15]
Test: [15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15]
```

6. Define Image Transforms

Define transforms with ImageNet normalization (required for pretrained models).

```
# ImageNet normalization values (REQUIRED for pretrained models)
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

# Define transforms
transform = transforms.Compose([
    transforms.Resize((224, 224)), # ResNet expects 224x224 input
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
])

print(f"Transform pipeline created:")
print(f"  - Resize to 224x224")
print(f"  - Convert to tensor")
print(f"  - Normalize with ImageNet mean={IMAGENET_MEAN}")
print(f"                                std={IMAGENET_STD}")
```

Transform pipeline created:

```
- Resize to 224x224
- Convert to tensor
- Normalize with ImageNet mean=[0.485, 0.456, 0.406]
                                std=[0.229, 0.224, 0.225]
```

7. Create Custom Dataset Class

Create a PyTorch Dataset class for loading images.

```
class UCMercedDataset(Dataset):
    """Custom Dataset for UCMerced Land Use images."""

    def __init__(self, image_paths, labels, transform=None):
        """
        Args:
            image_paths: List of paths to images
            labels: List of encoded labels
            transform: Optional transform to be applied
        """
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Load image
        img_path = self.image_paths[idx]
        image = Image.open(img_path).convert('RGB')

        # Apply transforms
        if self.transform:
            image = self.transform(image)

        label = self.labels[idx]
        return image, label

# Create dataset instances
```

```

train_dataset = UCMercedDataset(X_train, y_train, transform=transform)
val_dataset = UCMercedDataset(X_val, y_val, transform=transform)
test_dataset = UCMercedDataset(X_test, y_test, transform=transform)

print(f"Datasets created:")
print(f"  Train dataset: {len(train_dataset)} samples")
print(f"  Val dataset:   {len(val_dataset)} samples")
print(f"  Test dataset:  {len(test_dataset)} samples")

```

```

Datasets created:
  Train dataset: 1470 samples
  Val dataset:   315 samples
  Test dataset:  315 samples

```

8. Create DataLoaders

Create DataLoaders for batch processing during feature extraction.

```

# Create DataLoaders
BATCH_SIZE = 32

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=0)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=0)

print(f"DataLoaders created with batch_size={BATCH_SIZE}:")
print(f"  Train loader: {len(train_loader)} batches")
print(f"  Val loader:   {len(val_loader)} batches")
print(f"  Test loader:  {len(test_loader)} batches")

```

```

DataLoaders created with batch_size=32:
  Train loader: 46 batches
  Val loader:   10 batches
  Test loader:  10 batches

```

9. Load Pretrained ResNet as Feature Extractor

Load a pretrained ResNet50 model and remove the final classification layer to use it as a feature extractor.

```

# Load pretrained ResNet50
print("Loading pretrained ResNet50...")
resnet = models.resnet50(pretrained=True)

# Remove the final classification layer (fc layer)
# ResNet50's fc layer outputs 1000 classes (ImageNet)
# We want the features before this layer (2048-dimensional)
feature_extractor = nn.Sequential(*list(resnet.children())[:-1])

# Move to device and set to evaluation mode
feature_extractor = feature_extractor.to(device)
feature_extractor.eval()

# Freeze all parameters (no training needed)
for param in feature_extractor.parameters():
    param.requires_grad = False

print(f"\nFeature extractor created:")
print(f"  Model: ResNet50 (pretrained on ImageNet)")
print(f"  Output features: 2048-dimensional")
print(f"  Mode: Evaluation (frozen weights)")
print(f"  Device: {device}")

```

```

Loading pretrained ResNet50...
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/re
100%|██████████| 97.8M/97.8M [00:00<00:00, 169MB/s]

```

```

Feature extractor created:
  Model: ResNet50 (pretrained on ImageNet)
  Output features: 2048-dimensional
  Mode: Evaluation (frozen weights)
  Device: cuda

```

10. Extract Features from All Images

Extract features from train, validation, and test sets using the pretrained ResNet.

```
def extract_features(model, dataloader, device):
    """
    Extract features from images using the pretrained model.

    Args:
        model: Feature extractor model
        dataloader: DataLoader containing images
        device: Device to run on

    Returns:
        features: Numpy array of extracted features
        labels: Numpy array of labels
    """
    features = []
    labels = []

    model.eval()
    with torch.no_grad():
        for images, batch_labels in tqdm(dataloader, desc="Extracting features"):
            images = images.to(device)

            # Extract features
            batch_features = model(images)

            # Flatten to (batch_size, 2048)
            batch_features = batch_features.view(batch_features.size(0), -1)

            # Move to CPU and convert to numpy
            features.append(batch_features.cpu().numpy())
            labels.append(batch_labels.numpy())

    # Concatenate all batches
    features = np.vstack(features)
    labels = np.hstack(labels)

    return features, labels

# Extract features from all splits
print("Extracting features from training set...")
features_train, labels_train = extract_features(feature_extractor, train_loader, device)

print("\nExtracting features from validation set...")
features_val, labels_val = extract_features(feature_extractor, val_loader, device)

print("\nExtracting features from test set...")
features_test, labels_test = extract_features(feature_extractor, test_loader, device)

print(f"\nFeature extraction complete!")
print(f"  Train features: {features_train.shape}")
print(f"  Val features:   {features_val.shape}")
print(f"  Test features:  {features_test.shape}")
```

```
Extracting features from training set...
Extracting features: 100%|██████████| 46/46 [09:18<00:00, 12.13s/it]

Extracting features from validation set...
Extracting features: 100%|██████████| 10/10 [01:59<00:00, 11.97s/it]

Extracting features from test set...
Extracting features: 100%|██████████| 10/10 [02:02<00:00, 12.25s/it]
Feature extraction complete!
  Train features: (1470, 2048)
  Val features:   (315, 2048)
  Test features:  (315, 2048)
```

✓ 11. Train XGBoost Classifier

Now train an XGBoost classifier on the extracted features.

```
# Initialize XGBoost classifier
print("Training XGBoost classifier...")
xgb_classifier = XGBClassifier(
    n_estimators=200,
    max_depth=8,
    learning_rate=0.1,
    subsample=0.8,
```



```

        colsample_bytree=0.8,
        random_state=SEED,
        tree_method='hist', # Faster training
        eval_metric='mlogloss',
        verbosity=1
    )

# Train on extracted features
xgb_classifier.fit(
    features_train,
    labels_train,
    eval_set=[(features_val, labels_val)],
    verbose=True
)

print("\nXGBoost training complete!")

```

```

[144] validation_0-mlogloss:0.24225
[145] validation_0-mlogloss:0.24199
[146] validation_0-mlogloss:0.24186
[147] validation_0-mlogloss:0.24149
[148] validation_0-mlogloss:0.24128
[149] validation_0-mlogloss:0.24136
[150] validation_0-mlogloss:0.24147
[151] validation_0-mlogloss:0.24126
[152] validation_0-mlogloss:0.24120
[153] validation_0-mlogloss:0.24111
[154] validation_0-mlogloss:0.24102
[155] validation_0-mlogloss:0.24106
[156] validation_0-mlogloss:0.24092
[157] validation_0-mlogloss:0.24071
[158] validation_0-mlogloss:0.24049
[159] validation_0-mlogloss:0.24020
[160] validation_0-mlogloss:0.24007
[161] validation_0-mlogloss:0.24010
[162] validation_0-mlogloss:0.23980
[163] validation_0-mlogloss:0.23979
[164] validation_0-mlogloss:0.23969
[165] validation_0-mlogloss:0.23963
[166] validation_0-mlogloss:0.23927
[167] validation_0-mlogloss:0.23913
[168] validation_0-mlogloss:0.23908
[169] validation_0-mlogloss:0.23888
[170] validation_0-mlogloss:0.23871
[171] validation_0-mlogloss:0.23887
[172] validation_0-mlogloss:0.23891
[173] validation_0-mlogloss:0.23867
[174] validation_0-mlogloss:0.23873
[175] validation_0-mlogloss:0.23845
[176] validation_0-mlogloss:0.23812
[177] validation_0-mlogloss:0.23837
[178] validation_0-mlogloss:0.23835
[179] validation_0-mlogloss:0.23837
[180] validation_0-mlogloss:0.23831
[181] validation_0-mlogloss:0.23815
[182] validation_0-mlogloss:0.23813
[183] validation_0-mlogloss:0.23780
[184] validation_0-mlogloss:0.23771
[185] validation_0-mlogloss:0.23760
[186] validation_0-mlogloss:0.23761
[187] validation_0-mlogloss:0.23739
[188] validation_0-mlogloss:0.23729
[189] validation_0-mlogloss:0.23730
[190] validation_0-mlogloss:0.23729
[191] validation_0-mlogloss:0.23735
[192] validation_0-mlogloss:0.23727
[193] validation_0-mlogloss:0.23748
[194] validation_0-mlogloss:0.23751
[195] validation_0-mlogloss:0.23740
[196] validation_0-mlogloss:0.23741
[197] validation_0-mlogloss:0.23732
[198] validation_0-mlogloss:0.23724
[199] validation_0-mlogloss:0.23725

```

XGBoost training complete!

✓ 12. Evaluate on All Sets

Evaluate the classifier on train, validation, and test sets.

```

# Make predictions on all sets
y_train_pred = xgb_classifier.predict(features_train)
y_val_pred = xgb_classifier.predict(features_val)
y_test_pred = xgb_classifier.predict(features_test)

```



```
# Calculate metrics for each set
def evaluate_predictions(y_true, y_pred, set_name):
    """Calculate and print evaluation metrics."""
    accuracy = accuracy_score(y_true, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true, y_pred, average='weighted', zero_division=0
    )

    print(f"\n{set_name} Set Results:")
    print(f" Accuracy:  {accuracy:.4f}")
    print(f" Precision: {precision:.4f}")
    print(f" Recall:     {recall:.4f}")
    print(f" F1-Score:   {f1:.4f}")

    return accuracy, precision, recall, f1

# Evaluate all sets
train_metrics = evaluate_predictions(labels_train, y_train_pred, "Train")
val_metrics = evaluate_predictions(labels_val, y_val_pred, "Validation")
test_metrics = evaluate_predictions(labels_test, y_test_pred, "Test")
```

```
Train Set Results:
Accuracy:  1.0000
Precision:  1.0000
Recall:     1.0000
F1-Score:   1.0000

Validation Set Results:
Accuracy:   0.9270
Precision:  0.9285
Recall:     0.9270
F1-Score:   0.9263

Test Set Results:
Accuracy:   0.9270
Precision:  0.9316
Recall:     0.9270
F1-Score:   0.9271
```

13. Visualize Results

Create visualizations to better understand model performance.

```
# Confusion Matrix for Test Set (Raw Counts)
cm = confusion_matrix(labels_test, y_test_pred)

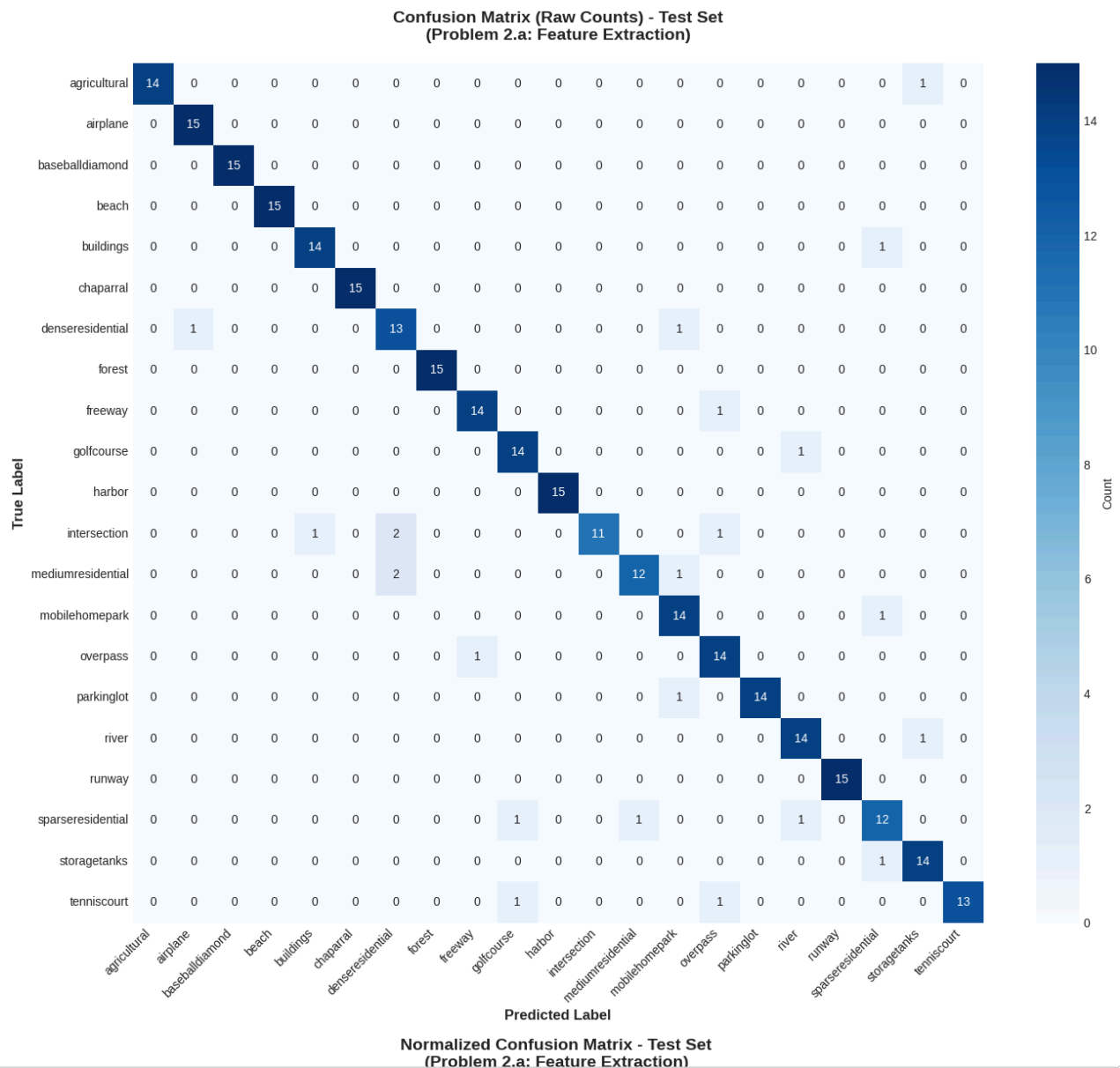
plt.figure(figsize=(14, 12))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names,
            cbar_kws={'label': 'Count'})
plt.title('Confusion Matrix (Raw Counts) - Test Set\n(Problem 2.a: Feature Extraction)',
          fontsize=14, fontweight='bold', pad=20)
plt.ylabel('True Label', fontsize=12, fontweight='bold')
plt.xlabel('Predicted Label', fontsize=12, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# Normalized Confusion Matrix (Required by assignment)
cm_normalized = confusion_matrix(labels_test, y_test_pred, normalize='true')

plt.figure(figsize=(14, 12))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names,
            cbar_kws={'label': 'Proportion'}, vmin=0, vmax=1)
plt.title('Normalized Confusion Matrix - Test Set\n(Problem 2.a: Feature Extraction)',
          fontsize=14, fontweight='bold', pad=20)
plt.ylabel('True Label', fontsize=12, fontweight='bold')
plt.xlabel('Predicted Label', fontsize=12, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

print("\nConfusion Matrix Statistics:")
print(f" Total samples: {cm.sum()}")
print(f" Correct predictions: {np.trace(cm)}")
```

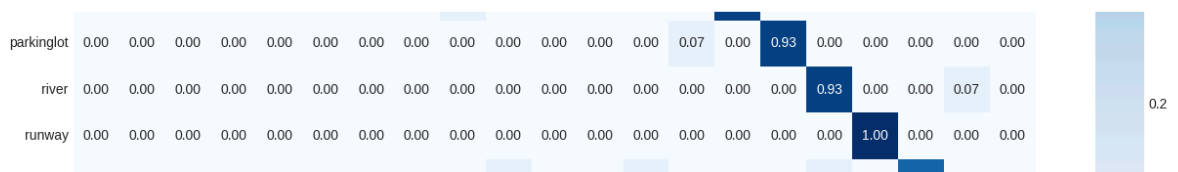
```
print(f" Overall accuracy: {np.trace(cm)/cm.sum():.4f}")
```

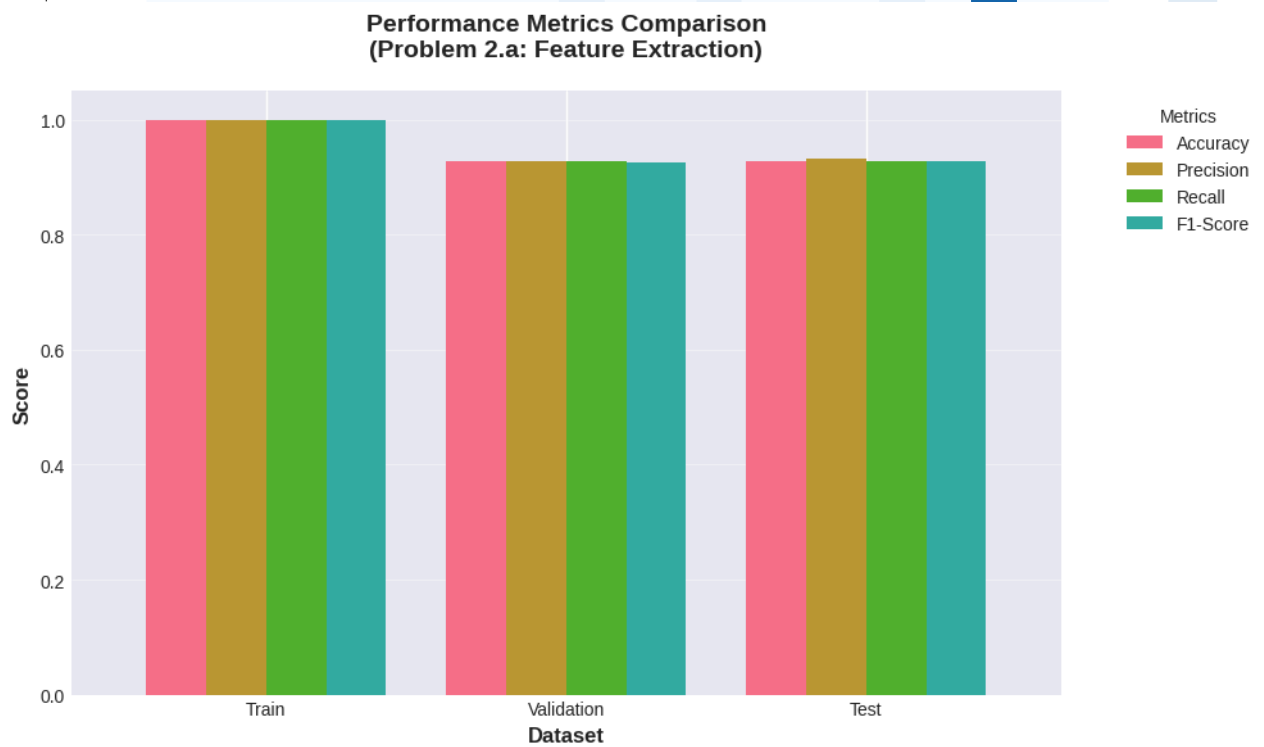



```
# Metrics Comparison Across Sets
metrics_df = pd.DataFrame({
    'Train': train_metrics,
    'Validation': val_metrics,
    'Test': test_metrics
}, index=['Accuracy', 'Precision', 'Recall', 'F1-Score'])

fig, ax = plt.subplots(figsize=(10, 6))
metrics_df.T.plot(kind='bar', ax=ax, width=0.8)
plt.title('Performance Metrics Comparison\n(Problem 2.a: Feature Extraction)',
         fontsize=14, fontweight='bold', pad=20)
plt.ylabel('Score', fontsize=12, fontweight='bold')
plt.xlabel('Dataset', fontsize=12, fontweight='bold')
plt.xticks(rotation=0)
plt.ylim([0, 1.05])
plt.legend(title='Metrics', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

# Print metrics table
print("\n" + "="*60)
print("SUMMARY TABLE")
print("="*60)
print(metrics_df.round(4))
print("="*60)
```





=====

SUMMARY TABLE

=====

	Train	Validation	Test
Accuracy	1.0000	0.9316	0.9316

```
# Per-Class Performance (Test Set)
print("\nDetailed Classification Report (Test Set):")
print("="*80)
print(classification_report(labels_test, y_test_pred,
                           target_names=class_names,
                           digits=4))
print("="*80)
```

Detailed Classification Report (Test Set):

	precision	recall	f1-score	support
agricultural	1.0000	0.9333	0.9655	15
airplane	0.9375	1.0000	0.9677	15
baseballdiamond	1.0000	1.0000	1.0000	15
beach	1.0000	1.0000	1.0000	15
buildings	0.9333	0.9333	0.9333	15
chaparral	1.0000	1.0000	1.0000	15
denseresidential	0.7647	0.8667	0.8125	15
forest	1.0000	1.0000	1.0000	15
freeway	0.9333	0.9333	0.9333	15
golfcourse	0.8750	0.9333	0.9032	15
harbor	1.0000	1.0000	1.0000	15
intersection	1.0000	0.7333	0.8462	15
mediumresidential	0.9231	0.8000	0.8571	15
mobilehomepark	0.8235	0.9333	0.8750	15
overpass	0.8235	0.9333	0.8750	15
parkinglot	1.0000	0.9333	0.9655	15
river	0.8750	0.9333	0.9032	15
runway	1.0000	1.0000	1.0000	15
sparseresidential	0.8000	0.8000	0.8000	15
storagetanks	0.8750	0.9333	0.9032	15
tenniscourt	1.0000	0.8667	0.9286	15
accuracy			0.9270	315
macro avg	0.9316	0.9270	0.9271	315
weighted avg	0.9316	0.9270	0.9271	315

=====

14. Per-Class Performance Visualization

Visualize precision, recall, and F1-score for each class.

```
# Calculate per-class metrics
precision_per_class, recall_per_class, f1_per_class, support = precision_recall_fscore_support(
    labels_test, y_test_pred, average=None, zero_division=0
)

# Create DataFrame for visualization
per_class_df = pd.DataFrame({
    'Class': class_names,
    'Precision': precision_per_class,
    'Recall': recall_per_class,
    'F1-Score': f1_per_class,
    'Support': support
})

# Sort by F1-score
per_class_df = per_class_df.sort_values('F1-Score', ascending=True)

# Plot
fig, ax = plt.subplots(figsize=(12, 10))
x = np.arange(len(per_class_df))
width = 0.25

bars1 = ax.barh(x - width, per_class_df['Precision'], width, label='Precision', alpha=0.8)
bars2 = ax.barh(x, per_class_df['Recall'], width, label='Recall', alpha=0.8)
bars3 = ax.barh(x + width, per_class_df['F1-Score'], width, label='F1-Score', alpha=0.8)

ax.set_xlabel('Score', fontsize=12, fontweight='bold')
ax.set_ylabel('Class', fontsize=12, fontweight='bold')
ax.set_title('Per-Class Performance Metrics (Test Set)\n(Problem 2.a: Feature Extraction)',
             fontsize=14, fontweight='bold', pad=20)
ax.set_yticks(x)
ax.set_yticklabels(per_class_df['Class'])
ax.legend()
ax.set_xlim([0, 1.05])
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

print("\nPer-Class Performance Summary:")
print(per_class_df.sort_values('F1-Score', ascending=False).to_string(index=False))
```